

Entregable 4. Colección de Postman exportada

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez | Docente: Juan Carlos Polania Cortes | Periodo 2026-2
| Repositorio: github.com/OrlandoRojasNa/taller02-rabbitmq-docker

Resumen

Archivo exportado: `postman/Taller02-RabbitMQ.postman_collection.json` (formato Postman Collection v2.1). Para usarla: Postman → *Import* → seleccionar el archivo.

Variable	Valor
<code>baseUrl</code>	<code>http://localhost:8080</code>
<code>rabbitUrl</code>	<code>http://localhost:15672</code>
<code>cola</code>	<code>cola.sensores</code>

10 mensajes válidos: *Runner* → marcar solo *Enviar mensaje valido* → *Iterations* = 10 → *Run*.
Resultado obtenido: 40/40 tests aprobados (figura 3 de este documento).

Peticiones

Enviar mensaje valido

POST `{{baseUrl}}/api/mensajes`

Cuerpo:

```
{
  "sensor": "0D-01",
  "valor": {{$randomInt}},
  "unidad": "mg/L",
  "iteracion": {{$randomInt}}
}
```

Tests:

```
pm.test('HTTP 200', () => pm.response.to.have.status(200));
pm.test('Content-Type application/json', () => pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json'));
const r = pm.response.json();
pm.test('status es boolean true', () =>
pm.expect(r.status).to.be.a('boolean').and.to.equal(true));
pm.test('message = Mensaje encolado', () => pm.expect(r.message).to.equal('Mensaje encolado'));
```

Enviar mensaje invalido (JSON mal formado)

POST `{{baseUrl}}/api/mensajes`

Cuerpo:

```
{
  "sensor": "OD-01",
  "valor": ,
  "unidad": "mg/L"
```

Tests:

```
pm.test('HTTP 400', () => pm.response.to.have.status(400));
pm.test('Content-Type application/json', () => pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json'));
const r = pm.response.json();
pm.test('status es boolean false', () =>
pm.expect(r.status).to.be.a('boolean').and.to.equal(false));
pm.test('message = Formato de mensaje invalido', () =>
pm.expect(r.message).to.equal('Formato de mensaje inválido'));
```

Enviar mensaje invalido (texto plano)

POST {{baseUrl}}/api/mensajes

Cuerpo:

```
esto no es un JSON
```

Tests:

```
pm.test('HTTP 400', () => pm.response.to.have.status(400));
const r = pm.response.json();
pm.test('status es boolean false', () =>
pm.expect(r.status).to.be.a('boolean').and.to.equal(false));
```

Estado de la cola (API de administracion RabbitMQ)

GET {{rabbitUrl}}/api/queues/%2F/{{cola}}

Tests:

```
const q = pm.response.json();
console.log(`cola=${q.name} durable=${q.durable} mensajes=${q.messages}
listos=${q.messages_ready} sin_ack=${q.messages_unacknowledged}
consumidores=${q.consumers}`);
pm.test('La cola es durable', () => pm.expect(q.durable).to.equal(true));
```

Ejecución

The screenshot shows a REST client interface with the following details:

- Overview:** POST Enviar mensaje valido
- URL:** `{{baseUrl}}/api/mensajes`
- Method:** POST
- Send Button:** Send
- Params:** Params, Authorization, Headers 10, Body, Scripts, Settings
- Query Params:** A table with columns Key, Value, Description, Bulk Edit, and a menu icon. The table contains one row with the headers Key, Value, and Description.
- Body:** Body, Cookies, Headers 5, Test Results 4/4, 200 OK, 347 ms, 208 B, Save Response
- JSON Response:**

```
1 {
2   "status": true,
3   "message": "Mensaje encolado"
4 }
```

Figura 1. Mensaje válido: 200 OK, {"status": true, "message": "Mensaje encolado"}. Tests 4/4.

Overview | POST Enviar mensaje valido | **POST Enviar mensaje invalido** | + | No Environment | Save | Share

Taller 02 - RabbitMQ en Docker > Enviar mensaje invalido (JSON mal formado)

POST | {{baseUrl}}/api/mensajes | Send

Docs | **Params** | Authorization | Headers 10 | Body | Scripts | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body | Cookies | Headers 4 | Test Results 4/4 | 400 Bad Request • 17 ms • 192 B • Save Response

{ } JSON | Preview | Debug with AI

```
1 {
2   "status": false,
3   "message": "Formato de mensaje inválido"
4 }
```

Figura 2. Mensaje inválido (JSON mal formado): 400 Bad Request, {"status": false, "message": "Formato de mensaje inválido"}. Tests 4/4.

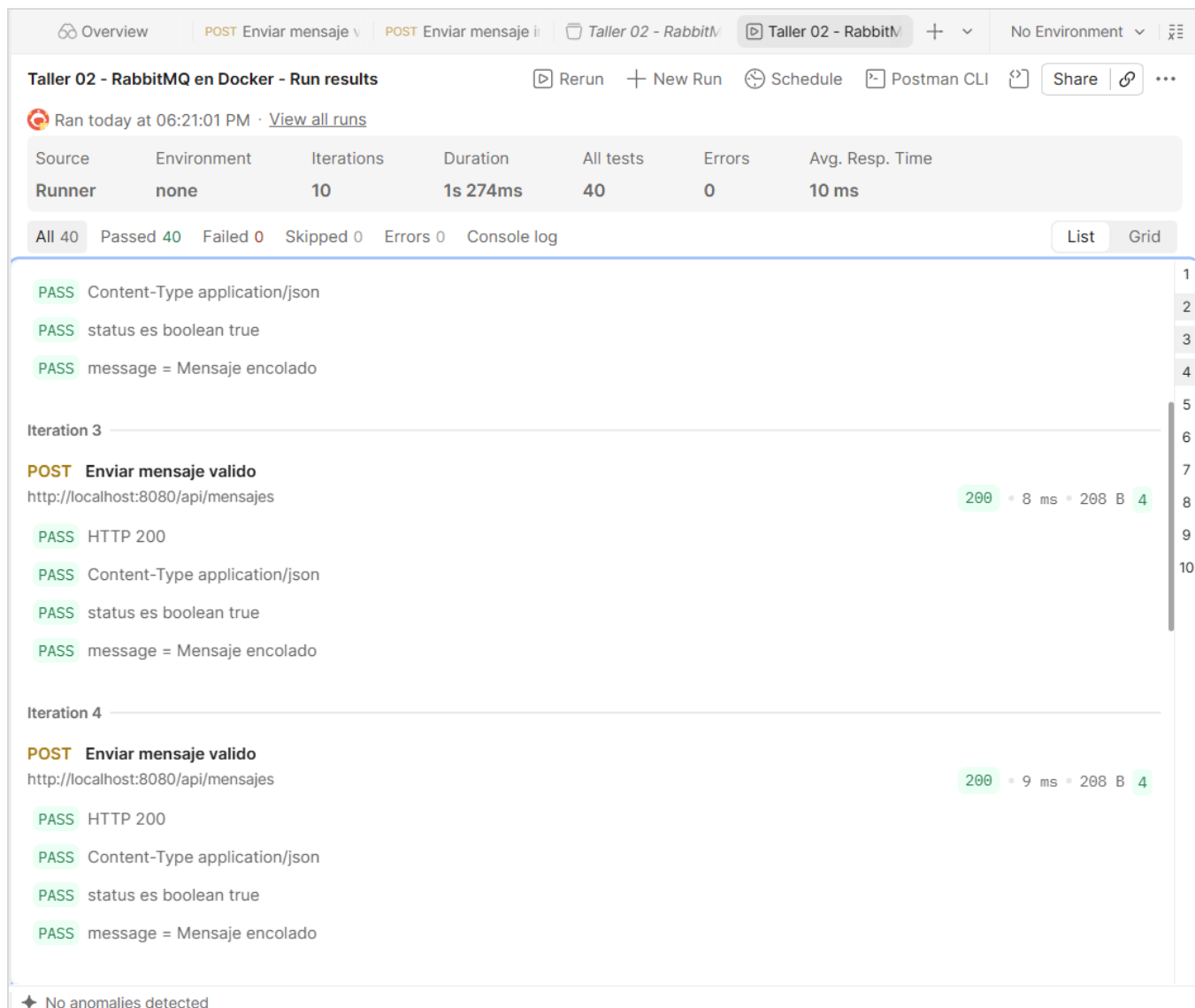


Figura 3. Collection Runner: 10 iteraciones de «Enviar mensaje valido». **40/40 tests aprobados, 0 errores.**

Colección exportada (JSON)

postman/Taller02-RabbitMQ.postman_collection.json

```
{
  "info": {
    "name": "Taller 02 - RabbitMQ en Docker",
    "description": "Pruebas del productor (API) y consulta del estado de la cola en la consola de RabbitMQ.\n\nPara enviar 10 mensajes validos: Runner -> seleccionar solo 'Enviar mensaje valido' -> Iterations = 10 -> Run.",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    { "key": "baseUrl", "value": "http://localhost:8080" },
    { "key": "rabbitUrl", "value": "http://localhost:15672" },
    { "key": "cola", "value": "cola.sensores" }
  ],
  "item": [
    {
      "name": "Enviar mensaje valido",
      "event": [
        {
          "listen": "test",
          "script": {
            "type": "text/javascript",
            "exec": [
              "pm.test('HTTP 200', () => pm.response.to.have.status(200));",
              "pm.test('Content-Type application/json', () => pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json'));",
              "const r = pm.response.json();",
              "pm.test('status es boolean true', () => pm.expect(r.status).to.be.a('boolean').and.to.equal(true));",
              "pm.test('message = Mensaje encolado', () => pm.expect(r.message).to.equal('Mensaje encolado'));"
            ]
          }
        }
      ],
      "request": {
        "method": "POST",
        "header": [
          { "key": "Content-Type", "value": "application/json" }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\n  \"sensor\": \"0D-01\",\n  \"valor\": {{ $randomInt }},\n  \"unidad\": \"mg/L\",\n  \"iteracion\": {{ $randomInt }}\n}",
          "options": { "raw": { "language": "json" } }
        },
        "url": {
          "raw": "{{ baseUrl }}/api/mensajes",
          "host": [ "{{ baseUrl }}" ],
          "path": [ "api", "mensajes" ]
        }
      },
      "response": []
    },
    {
      "name": "Enviar mensaje invalido (JSON mal formado)",
      "event": [
        {
          "listen": "test",
```

```

      "script": {
        "type": "text/javascript",
        "exec": [
          "pm.test('HTTP 400', () => pm.response.to.have.status(400));",
          "pm.test('Content-Type application/json', () =>
pm.expect(pm.response.headers.get('Content-Type')).to.include('application/json'));",
          "const r = pm.response.json();",
          "pm.test('status es boolean false', () =>
pm.expect(r.status).to.be.a('boolean').and.to.equal(false));",
          "pm.test('message = Formato de mensaje invalido', () =>
pm.expect(r.message).to.equal('Formato de mensaje inválido'));"
        ]
      }
    },
    ],
    "request": {
      "method": "POST",
      "header": [{ "key": "Content-Type", "value": "application/json" }],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"sensor\": \"0D-01\",\n  \"valor\": ,\n  \"unidad\": \"mg/L\"\n}"
      },
      "url": {
        "raw": "{{baseUrl}}/api/mensajes",
        "host": ["{{baseUrl}}"],
        "path": ["api", "mensajes"]
      }
    }
  },
  {
    "name": "Enviar mensaje invalido (texto plano)",
    "event": [
      {
        "listen": "test",
        "script": {
          "type": "text/javascript",
          "exec": [
            "pm.test('HTTP 400', () => pm.response.to.have.status(400));",
            "const r = pm.response.json();",
            "pm.test('status es boolean false', () =>
pm.expect(r.status).to.be.a('boolean').and.to.equal(false));"
          ]
        }
      }
    ],
    "request": {
      "method": "POST",
      "header": [{ "key": "Content-Type", "value": "text/plain" }],
      "body": { "mode": "raw", "raw": "esto no es un JSON" },
      "url": {
        "raw": "{{baseUrl}}/api/mensajes",
        "host": ["{{baseUrl}}"],
        "path": ["api", "mensajes"]
      }
    }
  },
  {
    "name": "Estado de la cola (API de administracion RabbitMQ)",
    "event": [
      {
        "listen": "test",

```

```
    "script": {
      "type": "text/javascript",
      "exec": [
        "const q = pm.response.json();",
        "console.log(`cola=${q.name} durable=${q.durable} mensajes=${q.messages} listos=${q.messages_ready} sin_ack=${q.messages_unacknowledged} consumidores=${q.consumers}`);",
        "pm.test('La cola es durable', () => pm.expect(q.durable).to.equal(true));"
      ]
    }
  },
  ],
  "request": {
    "auth": {
      "type": "basic",
      "basic": [
        { "key": "username", "value": "lab", "type": "string" },
        { "key": "password", "value": "lab123", "type": "string" }
      ]
    },
    "method": "GET",
    "header": [],
    "url": {
      "raw": "{{rabbitUrl}}/api/queues/%2F/{{cola}}",
      "host": ["{{rabbitUrl}}"],
      "path": ["api", "queues", "%2F", "{{cola}}"]
    }
  }
}
]
```